Práctica 1
Sistemas Distribuidos
Docker

José Antonio Mateo - David Muñoz – Jesús Jover

Instalación de Docker

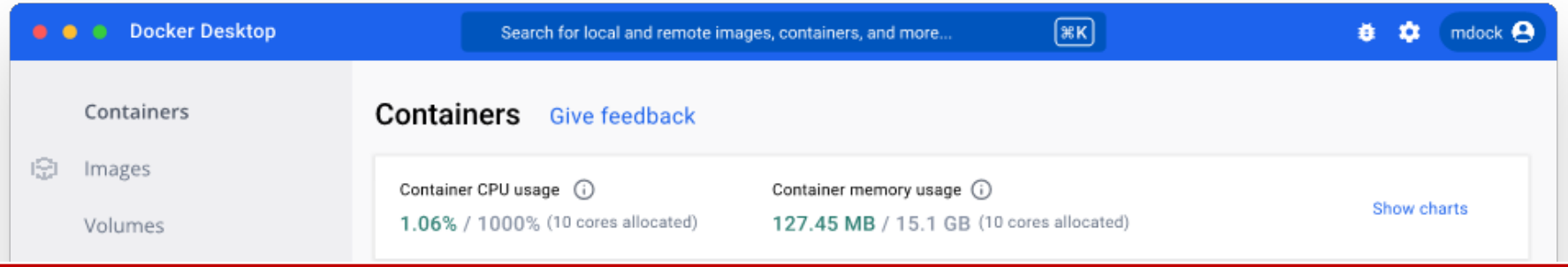
- En un sistema Linux

```
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
```

Hay más formas de instalarlo en: https://docs.docker.com/engine/install/ubuntu/

- En un sistema Windows / macOS: www.docker.com/products/docker-desktop/





01.

¿Qué son los contenedores?

Concepto

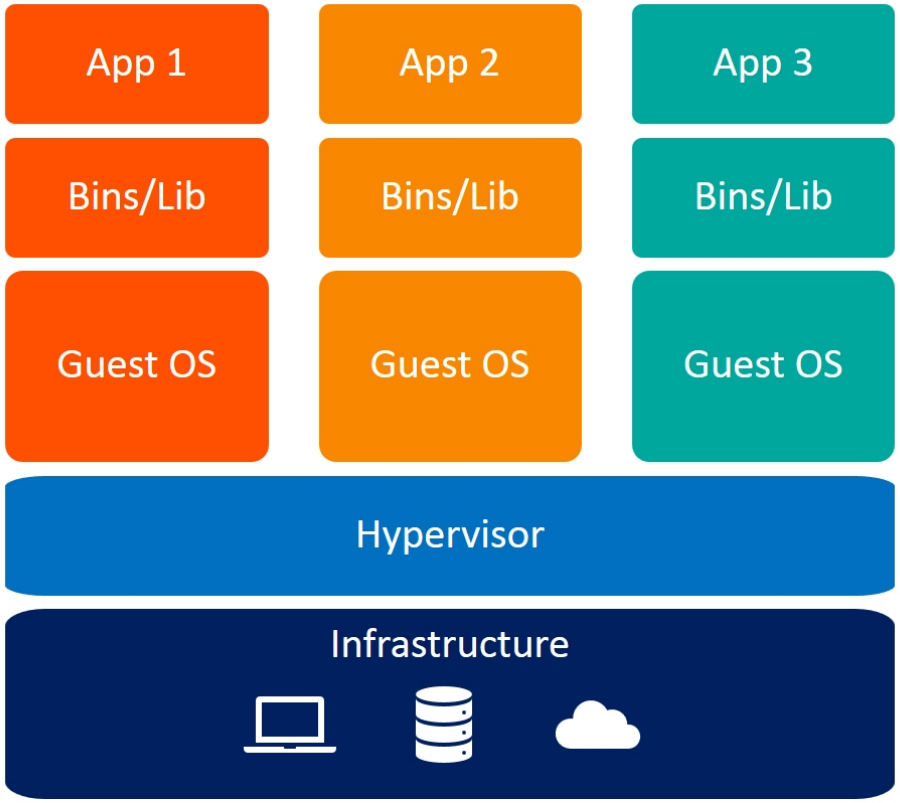
Paquete independiente de software con todo lo necesario para su ejecución (librerías, código, etc.)



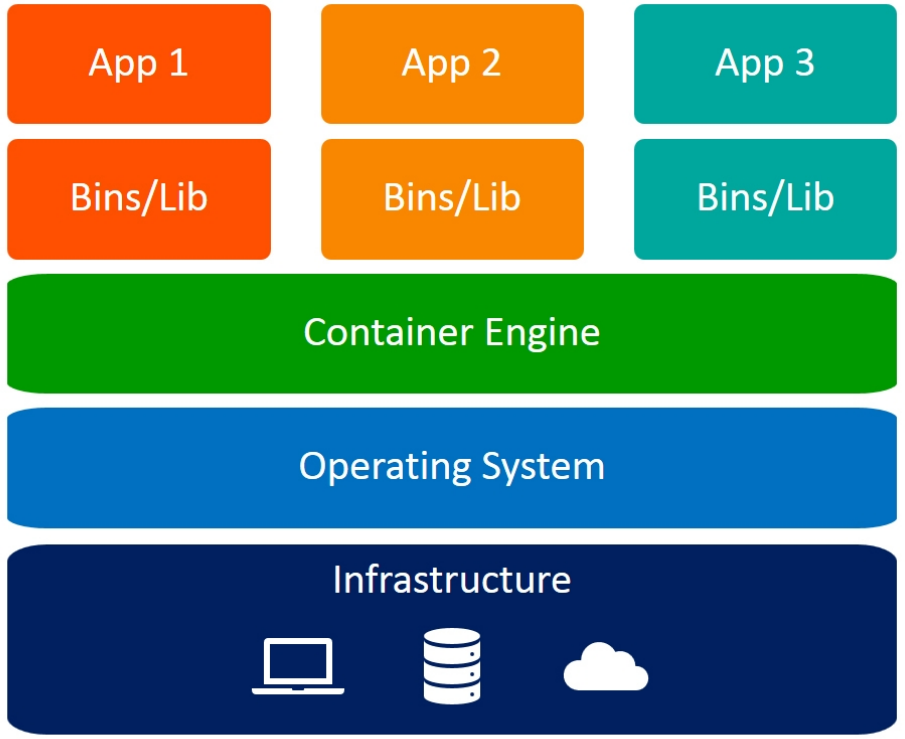
Características principales:

- Portabilidad
- Poca sobrecarga
- Aislamiento

Diferencia con las máquinas virtuales



Virtual Machines



Containers

Diferencia con las máquinas virtuales

Ventajas de los contenedores:

- **Menor consumo de recursos**, al usar una capa compartida y ser unidades mucho más pequeñas.
- **Inicio y parada más rápida**.
- **Mayor densidad por máquina**: una máquina puede alojar muchos más contenedores que máquinas virtuales.

Ambas ofrecen:

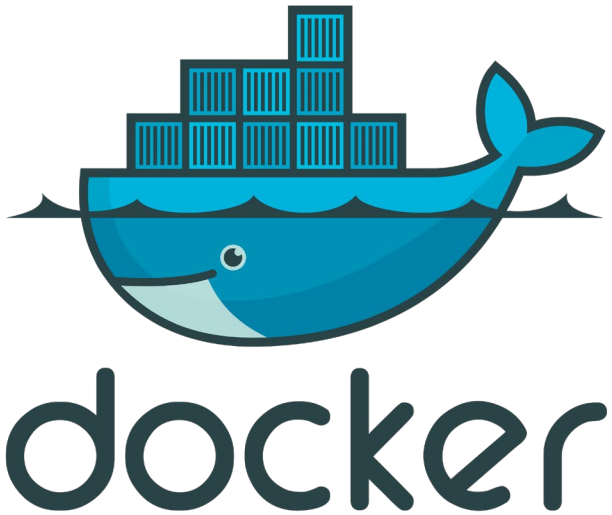
- **Aislamiento de recursos**.

02.
Docker

¿Qué es Docker?

Plataforma software que permite el despliegue de contenedores en multitud de sistemas operativos.

Es la plataforma de contenedores más extendida hoy en día.



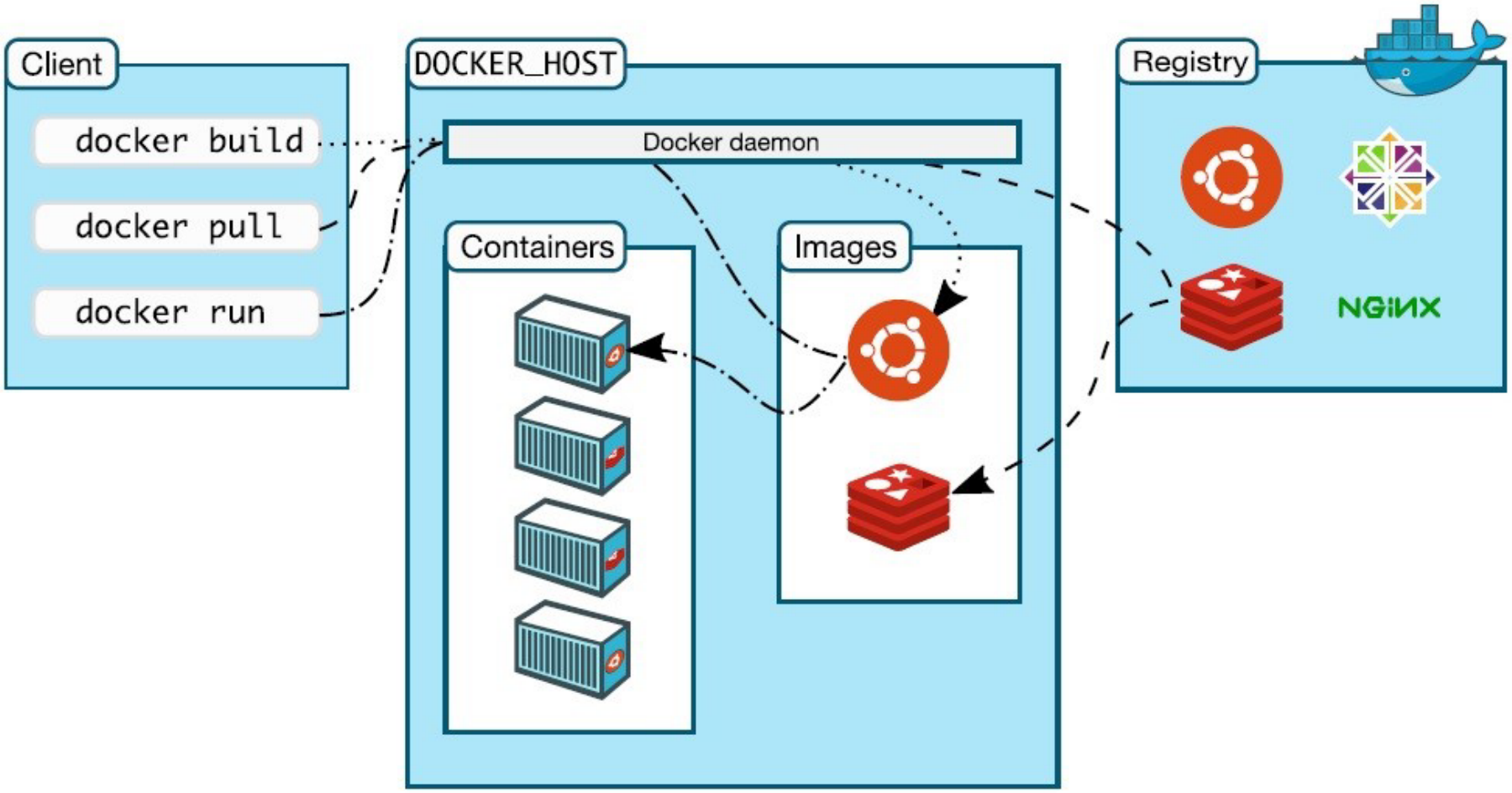
Características principales:

- Software libre
- Multiplataforma
- Estandarizado
- Gran catálogo de imágenes

Componentes de Docker

- **Docker Engine**: corazón de Docker. Se encarga de crear, gestionar y ejecutar los contenedores.
- **Docker Client**: herramienta CLI que permite a los usuarios interactuar con Docker Engine. Nos permite ordenar la creación, ejecución y parada de contenedores.
- **Docker Registry**: biblioteca de imágenes de contenedores de Docker.

Componentes de Docker



Ejecutando contenedores

- Verificar instalación de Docker

```
docker version
```

- Hola mundo en Docker

```
docker run hello-world
```

- Contenedor interactivo de ubuntu

```
docker run -it ubuntu
```

Ejecutando contenedores

```
docker run --name <nombre> --rm -t -i <imagen> <comando>
```

Algunas opciones:

Opción	¿Qué hace?
--name	Asigna un nombre al contenedor
-- rm	Elimina el contenedor al finalizar
-t	Permite el acceso mediante tty
-i	Acceso interactivo
- l	Permite asignar etiquetas al contenedor

Lista completa de opciones en https://docs.docker.com/reference/cli/docker/container/run/

Ver contenedores ejecutados

- Ver solo los contenedores activos

```
docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
c3163ab452cf	ubuntu	"/bin/bash"	9 seconds ago	Up 8 seconds		practical_ritchie

- Ver todos los contenedores

```
docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
1bb94ae27aa9	hello-world	"/hello“	4 seconds ago	Exited (0) 2 seconds ago		wonderful_leavitt
C3163ab452cf	ubuntu	"/bin/bash“	About an hour ago	Up About an hour		practical_ritchie

Parada de contenedores

- Parada de contenedor en ejecución de forma ordenada

```
docker stop <id-contenedor>
```



- Parada de contenedor en ejecución de forma forzosa

```
docker kill <id-contenedor>
```



- Parada y borrado de un contenedor de forma forzosa

```
docker rm -f <id-contenedor>
```



Ejercicio 1: stop vs. kill vs. rm -f

1. Ejecuta tres contenedores Ubuntu con **`docker run -it ubuntu &`**

```
...
...
...
```

2. Para el contenedor “*ubuntu1*” con stop

```
...
```

3. Para el contenedor “ubuntu2” con kill

```
...
```

4. Para el contenedor “ubuntu3” con rm -f

```
...
```

Ejercicio 1: stop vs. kill vs. rm -f

1. Ejecuta tres contenedores Ubuntu con una *bash*

```
docker run --name ubuntu1 -it ubuntu &
docker run --name ubuntu2 -it ubuntu &
docker run --name ubuntu3 -it ubuntu &
```

2. Para el contenedor "*ubuntu1*" con stop

```
docker stop ubuntu1
```

3. Para el contenedor "ubuntu2" con kill

```
docker kill ubuntu2
```

4. Para el contenedor "ubuntu3" con rm -f

```
docker rm -f ubuntu3
```

Pausa, inspección y estadísticas de los contenedores

- Pausa de un contenedor en ejecución

```
docker pause <id-contenedor>
docker unpause <id-contenedor>
```



- Acceso a los logs de un contenedor

```
docker logs [--timestamps] <id-contenedor>
```



- Estadísticas del uso de recursos del contenedor

```
docker stats <id-contenedor>
```



Imágenes

Imágenes: las recetas de los contenedores

Un contenedor se construye a partir de una imagen

```
docker run -it ubuntu
```

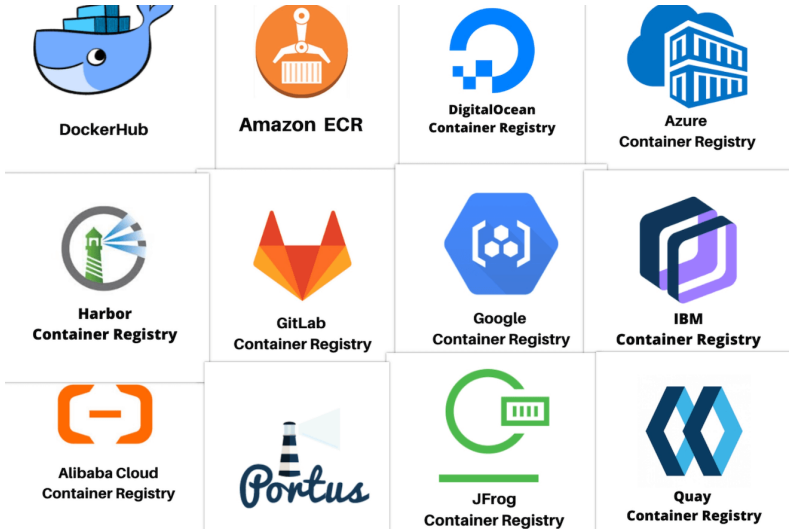


Una imagen es una plantilla de solo lectura que contiene toda la información para crear un contenedor

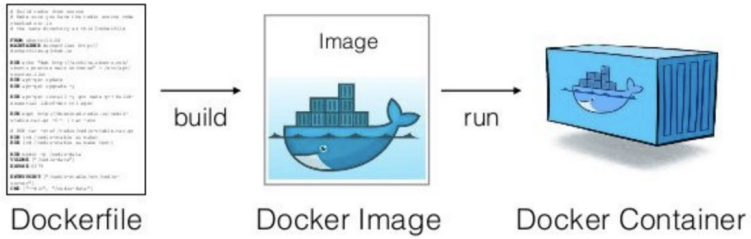
Imágenes: las recetas de los contenedores

¿De dónde salen las imágenes que usamos?

De un repositorio de imágenes

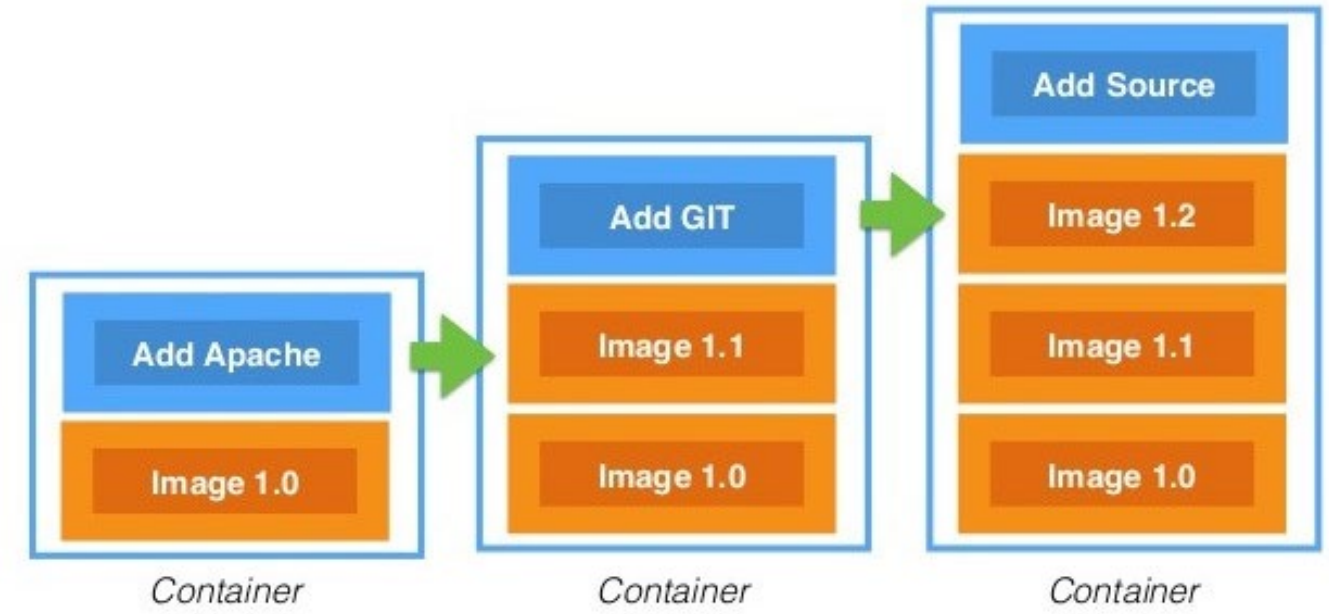


Las construimos nosotros



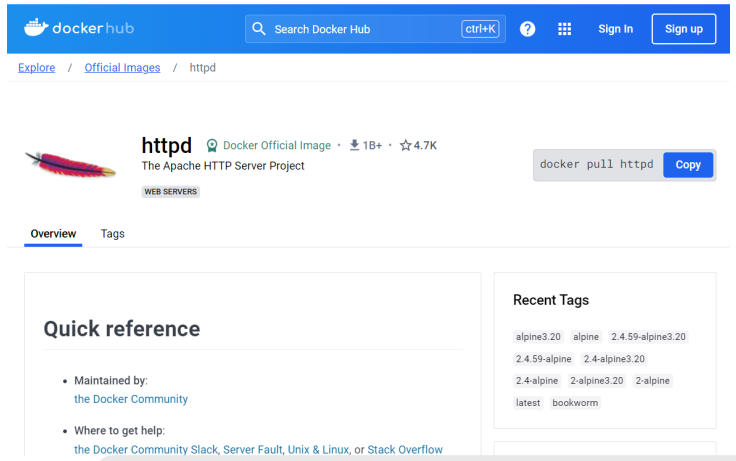
Imágenes: las recetas de los contenedores

Una imagen se construye a base de capas, cada vez que añadimos “algo”, añadimos una capa.



Imágenes: las recetas de los contenedores

La primera vez que usamos una imagen en un contenedor, si no está disponible localmente, se descargará de forma automática.




```
docker run httpd
```

```
pi@raspberrypi:~ $ docker run httpd
Unable to find image 'httpd:latest' locally
latest: Pulling from library/httpd
24c63b8dcb66: Pull complete
03b1da12ede7: Pull complete
4f4fb700ef54: Pull complete
f03286d1f6bc: Pull complete
54a622f797a4: Pull complete
a8f59b0044d9: Pull complete
Digest: sha256:43c7661a3243c04b0955c81ac994ea13a1d8a1e53c1
Status: Downloaded newer image for httpd:latest
```

Gestión de imágenes

- Para ver las imágenes disponibles localmente

```
docker image ls
```



- Para ver cómo se ha construido la imagen (capas)

```
docker image history <imagen>
```



- Para borrar una imagen localmente

```
docker image rm <imagen>
```

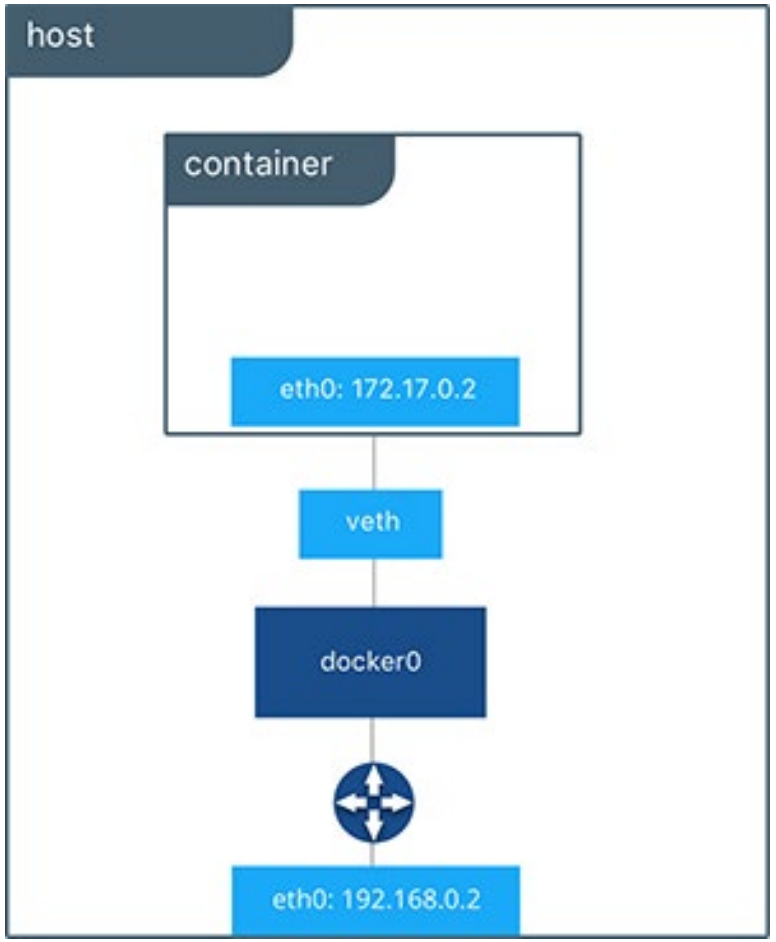


Redes

Redes en Docker

Docker conecta los contenedores a una red de tipo *bridge* por defecto.

Tendrán conectividad a la misma red donde esté conectado el host, pero **no podrán exponer ningún servicio por defecto.**

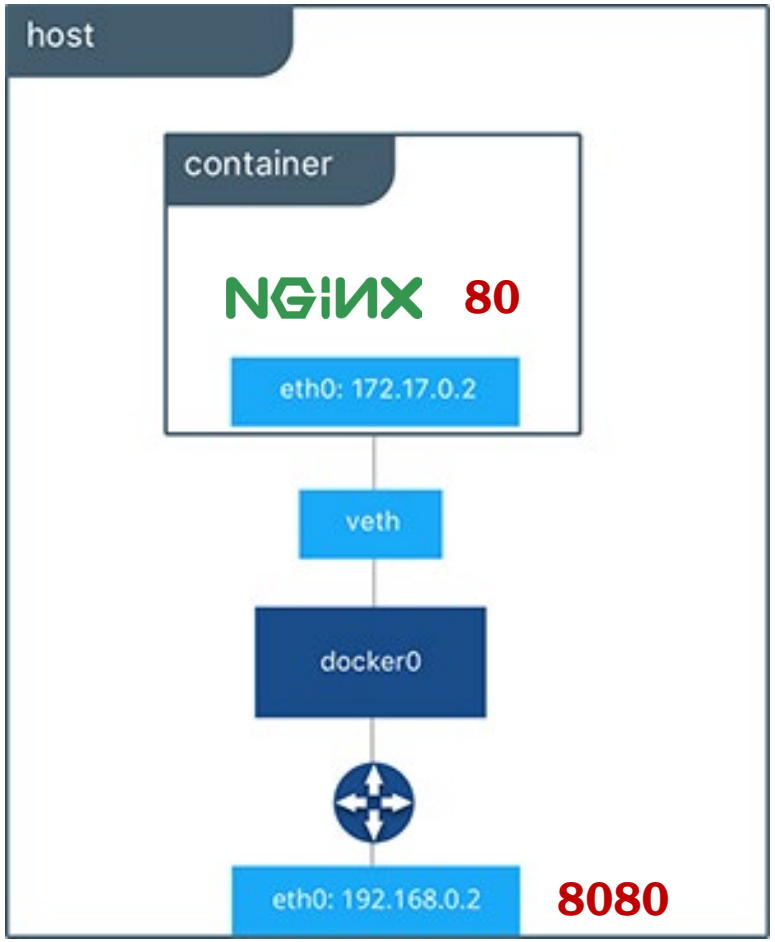


Mapeo de puertos

Si queremos exponer algún servicio, tenemos que **mapear puertos** (*port binding*)

```
docker run -p 8080:80 -d nginx
```

El puerto 8080 del host está siendo "enrutado" al puerto 80 del contenedor, donde hay instalado un servidor nginx.



Redes en Docker

También podemos crear **redes virtuales internas** para conectar contenedores entre sí. Hay **tres modos**:

- **Modo *bridge***: modo por defecto. El contenedor recibe una IP privada virtual de la red bridge.
- **Modo host**: se usa la misma interfaz de red y dirección IP del host, y permite exponer cualquier servicio desde el contenedor. Alto rendimiento, pero posibles riesgos de seguridad.
- **Modo *None***: aislamiento y desconexión de red total entre el host y los contenedores

Redes en Docker

- Para crear una nueva red

```
docker network create <nombre-red> -d [brigde/host/none]
```



- Ver las redes creadas

```
docker network ls
```



- Conectar una red a un contenedor (durante su creación)

```
docker run --net=<nombre-red> … <imagen>
```



- Conectar una red a un contenedor (después su creación)

```
docker network connect <nombre-red> <contenedor>
```



Ejercicio 2: comunicación entre contenedores

1. Crea una red en Docker llamada “red-ej2”

```
...
```

2. Crea un contenedor llamado “host1”, imagen “alpine” y conéctalo a la red

```
...
```

3. Crea un contenedor con imagen “alpine” y desde él haz ping al “host1”

```
...
```

Ejercicio 2: comunicación entre contenedores

1. Crea una red en Docker llamada “red-ej2”

```
docker network create red-ej2
```

2. Crea un contenedor llamado “host1”, imagen “alpine” y conéctalo a la red

```
docker run --name host1 --net=red-ej2 -itd alpine
```

3. Crea un contenedor con imagen “alpine” y desde él haz ping al “host1”

```
docker run --name host2 --net=red-ej2 alpine ping host1
```

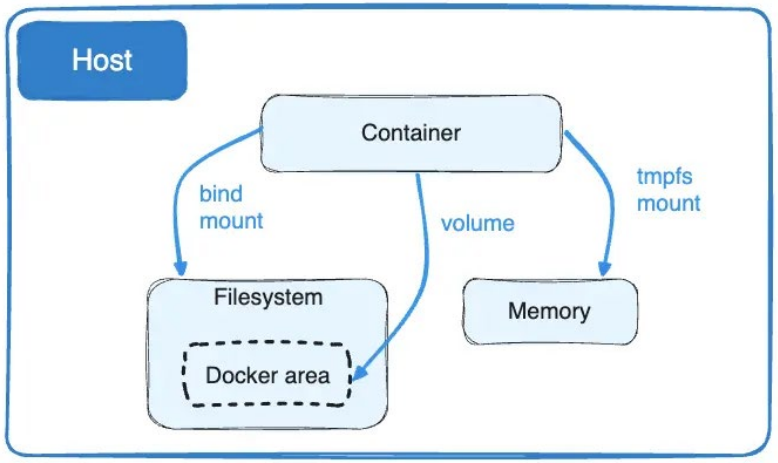
¿Por qué funciona el ping sin indicar la dirección IP?

```
cat /etc/resolv.conf
```

Almacenamiento

Almacenamiento en Docker

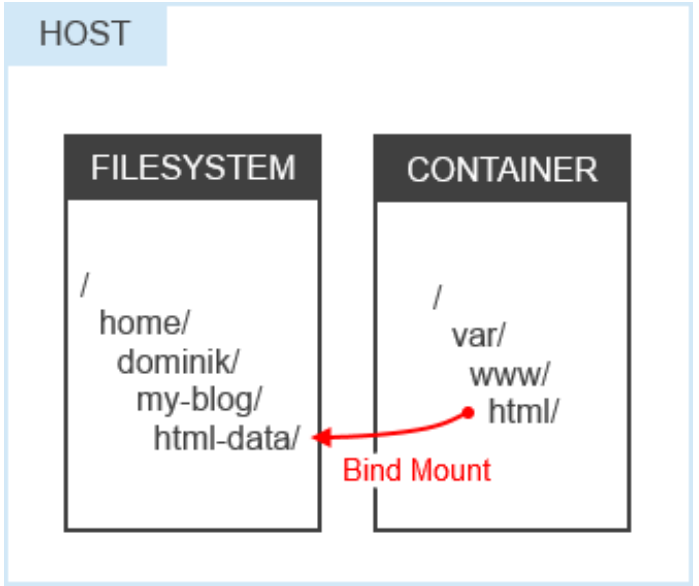
Los datos dentro de un contenedor se **eliminan** cuando el contenedor se elimina.



Mapeo de carpetas (*bind mount*)

Consiste en “compartir” una carpeta del host con el contenedor, permitiendo la visualización y edición en ambos sentidos.

```
docker run -v <dir_local>:<dir_cont> <imagen>
```



Ejercicio 3: servidor web con Apache

1. Crea un contenedor con la imagen “httpd”, mapeo de puertos 8080:80, mapeo de directorios de forma que el contenido del directorio “ej3” del material se monte en el directorio *“/usr/local/apache2/htdocs/”* del contenedor.

```
...
```

2. Accede a http://localhost:8080 desde el navegador.

3. Edita el contenido del fichero “index.html”, cambiando por ejemplo el título de la página.

4. Vuelve al navegador y actualiza la página. ¿Ha cambiado el contenido?

Ejercicio 3: servidor web con Apache

1. Crea un contenedor con la imagen “httpd”, mapeo de puertos 8080:80, mapeo de directorios de forma que el contenido del directorio “ej3” del material se monte en el directorio *“/usr/local/apache2/htdocs/”* del contenedor.

```
docker run -v .:/usr/local/apache2/htdocs/ -p 8080:80 httpd
```

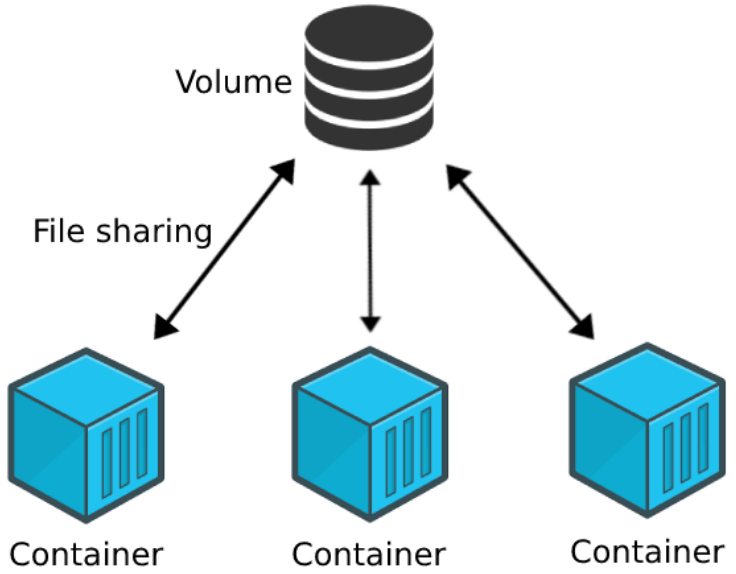
2. Accede a http://localhost:8080 desde el navegador.

3. Edita el contenido del fichero “index.html”, cambiando por ejemplo el título de la página.

4. Vuelve al navegador y actualiza la página. ¿Ha cambiado el contenido?

Volúmenes

Los volúmenes son “discos duros virtuales” que podemos conectar a un contenedor a través de un punto de montaje.



Gestión de volúmenes

- Para crear un nuevo volumen

```
docker volume create <nombre-vol>
```



- Ver los volúmenes creados

```
docker volume ls
```



- Imprime información sobre un volumen

```
docker volume inspect <nombre-vol>
```



- Conectar un volumen a un contenedor (durante su creación)

```
docker run -v <nombre-vol>:<punto-mont> … <imagen>
```



Ejercicio 4: servidor web con Apache

Validado DD

1. Crea un volumen de nombre “data_vol”

```
…
```

2. Crea un contenedor con la imagen “httpd”, mapeo de puertos 8081:80, y donde el volumen recién creado esté conectado a *“/usr/local/apache2/htdocs/”* del contenedor.

```
…
```

3. Accede a http://localhost:8081 desde el navegador ¿Qué se muestra?

It works!

4. Crea un contenedor interactivo con la imagen Alpine que monte el volumen creado en “/datos”, y edita el contenido del fichero “index.html”.

```
…
cd datos
echo “HOLA MUNDO DESDE EL CONTENEDOR ALPINE” > index.html
```

HOLA MUNDO DESDE EL CONTENEDOR ALPINE

Ejercicio 4: servidor web con Apache

1. Crea un volumen de nombre “data_vol”

```
docker volume create data_vol
```

2. Crea un contenedor con la imagen “httpd”, mapeo de puertos 8081:80, y donde el volumen recién creado esté conectado a *“/usr/local/apache2/htdocs/”* del contenedor.

```
docker run –v data_vol:/usr/local/apache2/htdocs/ -p 8081:80 httpd
```

3. Accede a http://localhost:8081 ¿Qué se muestra?

It works!

4. Crea un contenedor interactivo con la imagen Alpine que monte el volumen creado en “/datos”, y edita el contenido del fichero “index.html”.

```
docker run -it -v data_vol:/datos alpine
cd datos
echo “HOLA MUNDO DESDE EL CONTENEDOR ALPINE” > index.html
```

HOLA MUNDO DESDE EL CONTENEDOR ALPINE

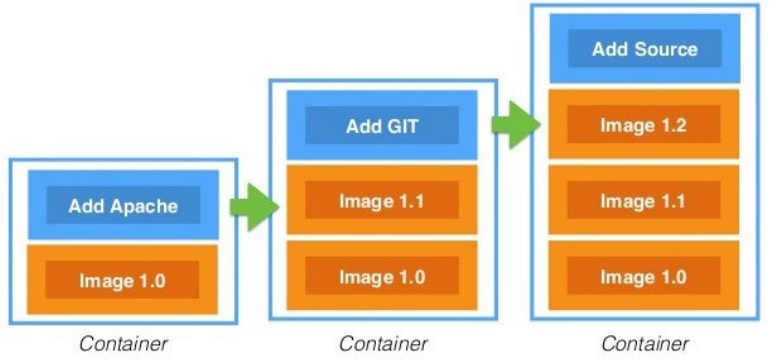
03.

Dockerfile

Creación de imágenes con Dockerfile

Si queremos ejecutar software propio en un contenedor necesitamos una imagen.





Ejemplo de Dockerfile con Apache

Dockerfile

```
FROM ubuntu

LABEL maintainer="SSDD"

RUN apt-get update &&
    apt-get install -y apache2 &&
    apt-get clean

COPY index.html /var/www/html

EXPOSE 80

CMD ["apachectl", "-D", "FOREGROUND"]
```

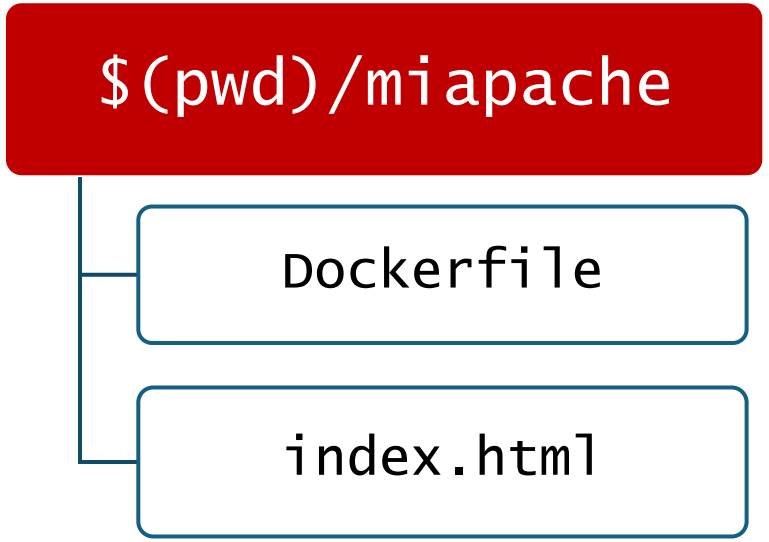


Imagen que incluye un servidor apache2 sobre Ubuntu, más mi página web principal

Construcción de la imagen a partir de Dockerfile

Importante escribir el punto

Construcción de la imagen desde el directorio actual

```
docker build -t <usuario-dockerhub>/miapache .
```

Uso de la imagen para poner en marcha un contenedor

```
docker run -p 8080:80 <usuario-dockerhub>/miapache
```

Principales comandos en Dockerfile

Builder main commands

command	description
FROM *image*|scratch	base image for the build
MAINTAINER *email*	name of the mainainer (metadata)
COPY *path dst*	copy *path* from the context into the container at location *dst*
ADD *src dst*	same as COPY but untar archives and accepts http urls
RUN *args...*	run an arbitrary command inside the container
USER *name*	set the default username
WORKDIR *path*	set the default working directory
CMD *args...*	set the default command
ENV *name value*	set an environment variable

Ejemplo:
CMD ["cmd", "-a", "arga-value", "-b", "argb-value"]



Instruction	Description
ADD	Add local or remote files and directories.
ARG	Use build-time variables.
CMD	Specify default commands.
COPY	Copy files and directories.
ENTRYPOINT	Specify default executable.
ENV	Set environment variables.
EXPOSE	Describe which ports your application is listening on.
FROM	Create a new build stage from a base image.
HEALTHCHECK	Check a container's health on startup.
LABEL	Add metadata to an image.
MAINTAINER	Specify the author of an image.
ONBUILD	Specify instructions for when the image is used in a build.
RUN	Execute build commands.
SHELL	Set the default shell of an image.
STOPSIGNAL	Specify the system call signal for exiting a container.
USER	Set user and group ID.
VOLUME	Create volume mounts.
WORKDIR	Change working directory.

Ejercicio 5: creando una imagen para una app

1. Genera el Dockerfile para una app propia.
   - Con la imagen base “ubuntu”
   - Actualiza los repositorios de Ubuntu:
     `apt update`
   - Instala el servidor apache2
     `apt install –y apache2`
   - Copia el contenido de “*ej5*” en *“/var/www/html”*
   - Expón el puerto 80
   - Ejecuta el comando `apachectl –D FOREGROUND`

2. Construye la imagen con el nombre “<tusiniciales>/miapp”

```
…
```

3. Crea un contenedor con ella mapeando los puertos 8083:80.

```
…
```

Ejercicio 5: creando una im

1. Genera el Dockerfile para una app propia
   - Con la imagen base “ubuntu”
   - Actualiza los repositorios de Ubuntu: `apt update`
   - Instala el servidor apache2 `apt install -y apache2`
   - Copia el contenido de “ej5” en “/var/
   - Expón el puerto 80
   - Ejecuta el comando `apachectl -D F`

Dockerfile

```
FROM ubuntu
LABEL maintainer="jja"

RUN apt update
RUN apt install -y apache2

COPY index.html /var/www/html/index.html
COPY assets /var/www/html/assets

EXPOSE 80

CMD ["apachectl", "-D", "FOREGROUND"]
```

2. Construye la imagen con el nombre

```
docker build -t jja/miapp .
```

3. Crea un contenedor con ella mapeando los puertos 8083:80.

```
docker run -d -p 8083:80 jja/miapp
```

04.

docker-compose

Despliegues con docker-compose

¿Qué ocurre si para ejecutar mi aplicación/servicio necesito ejecutar 3 contenedores con 2 volúmenes y una red entre ellos:

```
docker volume create vol1
docker volume create vol2

docker network create net1 -d bridge

docker run -v vol1:/data -p 80:8080 imagen1
docker run -v vol1:/data -p 21:2100 imagen2
docker run -v vol1:/data -p 9090:9090 imagen3
```

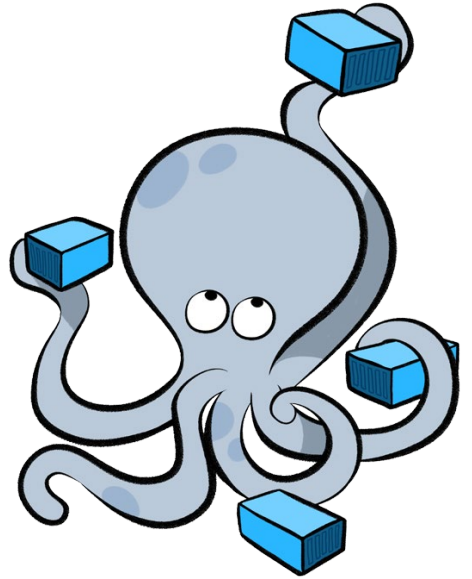
Despliegues con docker-compose

¿Qué ocurre si para ejecutar mi aplicación/servicio necesito ejecutar 3 contenedores con 2 volúmenes y una red entre ellos:

Imposible de mantener

```
docker run -v vol1:/data -p 80:8080 imagen1
docker run -v vol1:/data -p 21:2100 imagen2
docker run -v vol1:/data -p 9090:9090 imagen3
```

Despliegues con docker-compose



docker-compose nos permite “redactar” un despliegue complejo de Docker a través de un fichero.

Ventajas principales:

- Pasamos de decenas de comandos a un solo fichero
- Permite definir el estado final, no los pasos
- Ejecución, parada y borrado con un solo comando
- Podemos controlar las versiones con Git

Ejemplo docker-compose

docker-compose.yml

```
services:
  frontend:
    image: my-app
    ports:
         - “80:8080”
    volumes:
         - ./miapp:/app
         - data:/datos
    networks:
         - miapp-net
  db:
    image: postgres
    ports:
         - “5432:5432”
    networks:
         - miapp-net

volumes:
  data:
networks:
  miapp-net:
    driver: bridge
```

Ejemplo docker-compose

docker-compose.yml

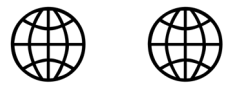
```
services:
  frontend:
    image: my-app
    ports:
        - “80:8080”
    volumes:
        - ./miapp:/app
        - data:/datos
    networks:
        - miapp-net
  db:
    image: postgres
    ports:
        - “5432:5432”
    networks:
        - miapp-net

volumes:
  data:
networks:
  miapp-net:
    driver: bridge
```

- Nombres de los contenedores
- Mapeo de puertos
- Mapeo de directorios
- Red de Docker
- Volumen de Docker

* No es estrictamente necesario crear una red ya que se crea por defecto

Existen muchas opciones de configuración:



Gestión de despliegues

- Realizar el despliegue (levantar contenedores, volúmenes, etc.)

```
docker compose up -d
```



- Para detener un despliegue realizado

```
docker compose stop
```



- Para borrar los elementos del despliegue (excepto volúmenes)

```
docker compose down
```



- Si solo queremos borrar los contenedores parados

```
docker compose rm
```



05.
Orquestación de contenedores

¿Qué es la orquestación?

La **orquestación** permite la automatización, gestión, despliegue y escalado de aplicaciones basadas en contenedores.

Permite que las aplicaciones se ejecuten de manera fiable y eficiente en entornos distribuidos como en clústeres de servidores.



Ventajas de la orquestación

- **Escalabilidad**: con un orquestador de contenedores podemos aumentar o reducir de forma automática el número de recursos según la demanda.
- **Alta disponibilidad**: garantiza que los contenedores siempre estén disponibles, incluso si la máquina en el que está falla.
- **Gestión eficiente**: nos permite actualizar las aplicaciones poco a poco.
- **Portabilidad**: permite ejecutar aplicaciones en diferentes entornos sin modificarlas.

Kubernetes

El orquestador más popular y usado actualmente. Fue creado por Google en 2014, y después donado para hacerlo de código libre.



kubernetes

Práctica 1
Sistemas Distribuidos
Docker

José Antonio Mateo - David Muñoz – Jesús Jover